

Computer Networks Assignment Report

Extending Echo Client/Server Functions

Gunda Sai Yashaswith (ES24BTECH11015)

Kanhegaonkar Malhar Ganesh (ES24BTECH11018)

Sheshadri Rathan CS (ES24BTECH11030)

Problem Statement

Q1. Integrate `getaddrinfo()` into the client so that the server hostname can be given as a command-line option. Demonstrate successful hostname-based execution and a Wireshark/tcpdump capture proving a DNS query and response.

Q2. Add at least two features to the Echo Client/Server and demonstrate them. The report should describe the new features and their benefits.

Implemented Features

Q1 – Hostname support: Hostname resolution using `getaddrinfo()`, followed by TCP connection.

Feature 1 – Timestamps: Time displayed for important client/server network events.

Feature 2 – RTT: TCP round-trip delay measured in milliseconds.

Feature 3 – IPv6: TCP communication using the IPv6 loopback address.

Feature 4 – Multicasting: One server distributes messages to multiple clients through IPv4 and IPv6 multicast.

Q1. Hostname Support using getaddrinfo()

Summary: The client accepts a server hostname instead of requiring a numerical IP address. `getaddrinfo()` converts the hostname and port into usable socket addresses, which the client then tries until a connection succeeds.

Implementation in our code: The client sets `ai_family = AF_UNSPEC` and uses TCP settings in the `addrinfo` hints. It calls `getaddrinfo(serverName, serverPort, ...)` and loops through the returned addresses, creating a socket and calling `connect()` until successful.

Hostname used: 127.0.0.1.nip.io, with the command `./client 127.0.0.1.nip.io 12345`.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri Rathana CS/OneDrive/Documents/CN Assignment 1$ ./server 12345
[17:50:38] Server started
[17:50:38] TCP IPv4 listening on port 12345
[17:50:38] TCP IPv6 listening on port 12345
[17:50:38] IPv4 multicast: 239.1.1.1:5000
[17:50:38] IPv6 multicast: [ff02::1234]:5000
[17:50:46] Client connected: 127.0.0.1:57278
█
```

Figure 1. Echo server running on port 12345 and accepting the client connection.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri Rathana CS/OneDrive/Documents/CN Assignment 1$ ./client 127.0.0.1.nip.io 12345
[17:50:46] Connected to 127.0.0.1.nip.io:12345
█
```

Figure 2. Client successfully connects using the hostname 127.0.0.1.nip.io.

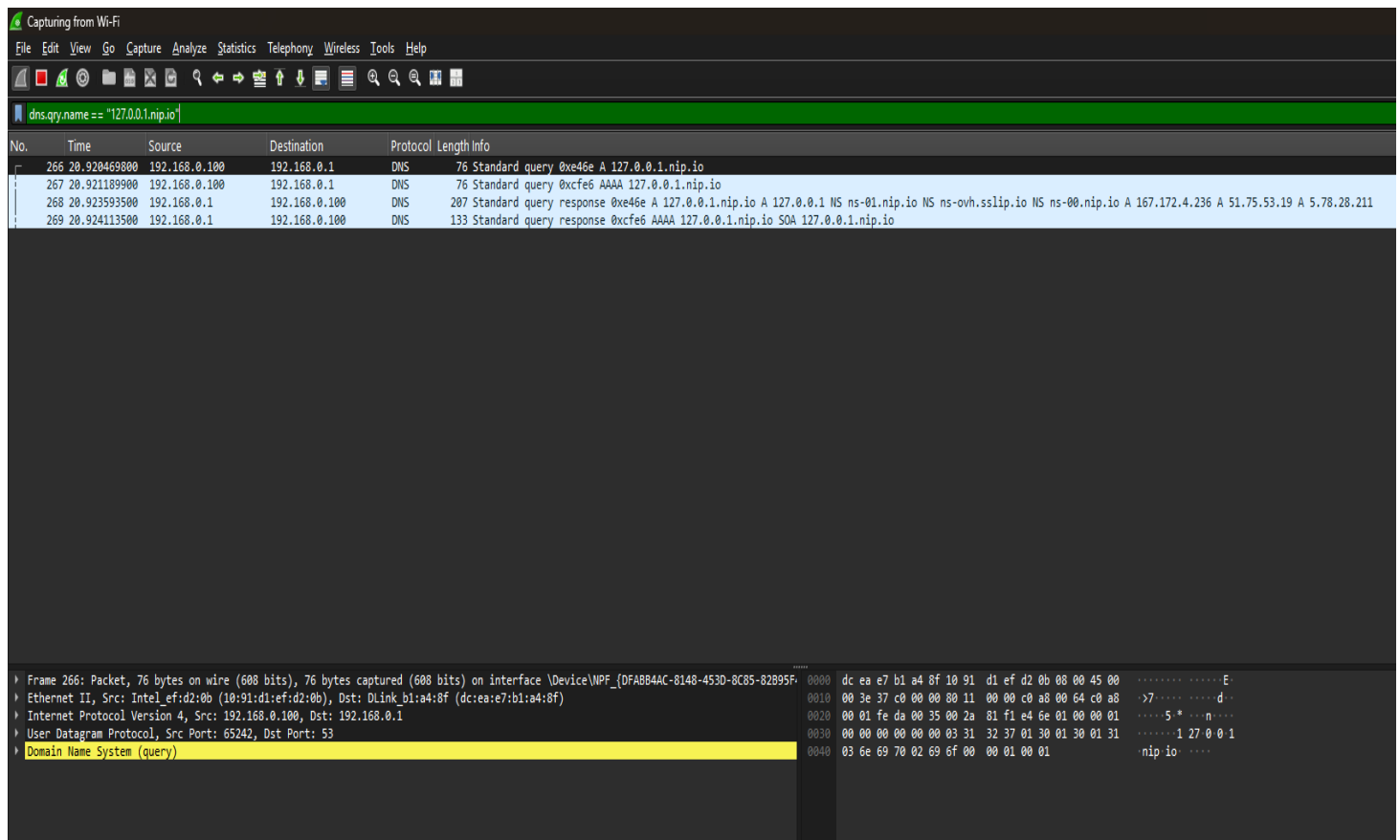


Figure 3. Wireshark capture showing the DNS query and response for 127.0.0.1.nip.io.

Additional Features

Feature 1: Timestamped Network Events

Summary: Timestamps are printed before important communication events, making the order of events easy to follow.

Implementation in our code: Both client and server use `printTimestamp()`, which obtains the current time with `time()`, converts it with `localtime()`, and prints `[HH:MM:SS]`.

Benefit: Makes communication logs clearer and helps with debugging and tracing network events.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri Rathana CS/OneDrive/Documents/CN Assignment 1$ ./server 12345
[17:50:38] Server started
[17:50:38] TCP IPv4 listening on port 12345
[17:50:38] TCP IPv6 listening on port 12345
[17:50:38] IPv4 multicast: 239.1.1.1:5000
[17:50:38] IPv6 multicast: [ff02::1234]:5000
[17:50:46] Client connected: 127.0.0.1:57278
[17:52:08] TCP received: Hi
[17:52:08] IPv4 multicast received: Hi
[17:52:08] IPv6 multicast received: Hi
```

Figure 4a. Server timestamps.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri Rathana CS/OneDrive/Documents/CN Assignment 1$ ./client 127.0.0.1.nip.io 12345
[17:50:46] Connected to 127.0.0.1.nip.io:12345
Hi
Type string: [17:52:08] TCP sent: Hi
[17:52:08] TCP received: Hi
RTT: 0.572 ms
[17:52:08] MULTICAST IPv4: Hi
[17:52:08] MULTICAST IPv6: Hi
```

Figure 4b. Client timestamps.

Feature 2: TCP Round-Trip Time (RTT)

Summary: The client measures the time from sending a TCP message until its echoed response is received.

Implementation in our code: The client records start before `send()` and end after `recv()` using `clock_gettime(CLOCK_MONOTONIC, ...)`. The difference is converted to milliseconds and printed.

Benefit: Gives a simple measure of network responsiveness and communication delay.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri Rathana CS/OneDrive/Documents/CN Assignment 1$ ./client 127.0.0.1.nip.io 12345
[17:50:46] Connected to 127.0.0.1.nip.io:12345
Hi
Type string: [17:52:08] TCP sent: Hi
[17:52:08] TCP received: Hi
RTT: 0.572 ms
[17:52:08] MULTICAST IPv4: Hi
[17:52:08] MULTICAST IPv6: Hi
```

Figure 5. Client showing the measured TCP RTT.

Additional Features (continued)

Feature 3: IPv6 Support

Summary: The Echo Client/Server supports TCP communication over IPv6 as well as IPv4.

Implementation in our code: The server creates a separate IPv6 TCP socket using `AF_INET6` and listens on the same port. The client can handle IPv6 results because `getaddrinfo()` uses `AF_UNSPEC`.

Benefit: Allows the application to work on IPv6 networks and communicate using IPv6 addresses.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri_Rathan_CS/OneDrive/Documents/CN Assignment 1$ ./client ::1 12345
[17:53:21] Connected to ::1:12345
Hi IPv6
Type string: [17:53:42] TCP sent: Hi IPv6
[17:53:42] TCP received: Hi IPv6
RTT: 0.248 ms
```

Figure 6. Client communicating using the IPv6 loopback address `::1`.

Feature 4: IPv4 and IPv6 Multicasting

Summary: A message received by the server is also distributed to multicast groups, allowing multiple clients to receive the same message.

Implementation in our code: The server creates IPv4 and IPv6 multicast sockets and sends each received TCP message to both groups. Clients join the groups and monitor them with `select()`.

Groups used: IPv4 `239.1.1.1:5000` and IPv6 `[ff02::1234]:5000`. The demonstration uses one server and two clients.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri_Rathan_CS/OneDrive/Documents/CN Assignment 1$ ./server 12345
[17:54:43] Server started
[17:54:43] TCP IPv4 listening on port 12345
[17:54:43] TCP IPv6 listening on port 12345
[17:54:43] IPv4 multicast: 239.1.1.1:5000
[17:54:43] IPv6 multicast: [ff02::1234]:5000
[17:54:55] Client connected: 127.0.0.1:33754
[17:55:26] TCP received: Hi from client 1
[17:55:26] IPv4 multicast received: Hi from client 1
[17:55:26] IPv6 multicast received: Hi from client 1
[17:55:32] Client connected: 127.0.0.1:36574
[17:55:39] TCP received: Hi from client 2
[17:55:39] IPv4 multicast received: Hi from client 2
[17:55:39] IPv6 multicast received: Hi from client 2
```

Figure 7a. Server receiving messages from client 1 and client 2 and forwarding them to both multicast groups.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri_Rathan_CS/OneDrive/Documents/CN Assignment 1$ ./client 127.0.0.1.nip.io 12345
[17:54:55] Connected to 127.0.0.1.nip.io:12345
Hi from client 1
Type string: [17:55:26] TCP sent: Hi from client 1
[17:55:26] TCP received: Hi from client 1
RTT: 0.256 ms
[17:55:26] MULTICAST IPv4: Hi from client 1
[17:55:26] MULTICAST IPv6: Hi from client 1
[17:55:39] MULTICAST IPv4: Hi from client 2
[17:55:39] MULTICAST IPv6: Hi from client 2
```

Figure 7b. Client 1 receiving multicast messages.

```
sheshadri_rathan@LAPTOP-2LCL07UT:/mnt/c/Users/Sheshadri_Rathan_CS/OneDrive/Documents/CN Assignment 1$ ./client 127.0.0.1.nip.io 12345
[17:55:32] Connected to 127.0.0.1.nip.io:12345
Hi from client 2
Type string: [17:55:39] TCP sent: Hi from client 2
[17:55:39] TCP received: Hi from client 2
RTT: 0.202 ms
[17:55:39] MULTICAST IPv4: Hi from client 2
[17:55:39] MULTICAST IPv6: Hi from client 2
█
```

Figure 7c. Client 2 receiving multicast messages.

Benefit: One sender can distribute the same information to multiple receivers efficiently.

References

1. **Mockapetris, P. (1987). Domain Names** — Implementation and Specification. RFC 1035, Internet Engineering Task Force (IETF).
2. **Donahoo, M. J., & Calvert, K. L. (2009). TCP/IP Sockets in C** —Practical Guide for Programmers (2nd ed.). Morgan Kaufmann.
3. **nip.io** — Hostname-to-IP address mapping service. <https://nip.io/>
4. **Linux man-pages project.** — getaddrinfo(3) ,Linux manual page: network address and service translation.
5. **Deering, S. E. (1989). Host Extensions for IP Multicasting.** RFC 1112, Internet Engineering Task Force (IETF).
6. **Hinden, R. M., & Deering, S. E. (2006). IP Version 6 Addressing Architecture.** RFC 4291, Internet Engineering Task Force (IETF).